

gt security guide — gt unlock

Written against **gt v0.20.0** (gt-lotr 0.3.0). Unlock ships **off**. This guide says what it does when you turn it on, what it needs, how to set it up on macOS, Windows and Linux, and — just as plainly — what it does not protect.

"Unlock proves a person was present and limits what agents can do on their own. It is not anti-malware. If something already runs as you, it can wait for you to unlock."

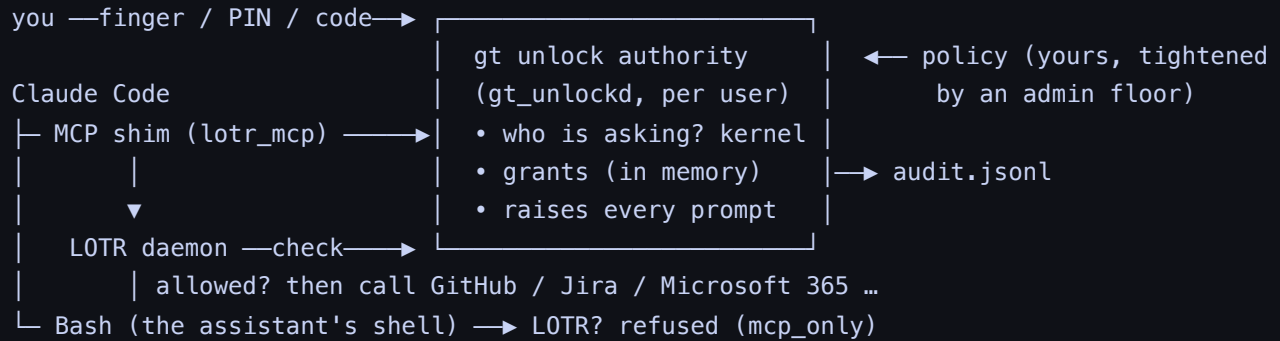
1. The goal, and what was done to get there

gt aims to make everything it touches as secure as it can, and to tell you exactly how secure that is on your machine. An AI agent working for you runs as you: it can call tools, run shell commands and read your files. gt unlock puts a person back in the loop for the things that matter — your connected systems (LOTR), your credentials, publishing, and gt's own guards — without slowing down ordinary work.

What was built, and why each piece is there:

MEASURE	WHAT IT STOPS
Kernel peer identity. Every caller is identified by the operating system (macOS audit token with the process version, Linux <code>S0_PEERCRED</code> + start time, Windows named-pipe client pid + creation time + user SID) — never by anything the caller says	A process pretending to be another one
Signatures, not booleans. Touch ID and Windows Hello sign a fresh challenge with a hardware key (Secure Enclave / TPM); gt verifies the signature itself in Python	A fake helper that just answers "ok"; a replayed old approval
Sealed credentials (<code>sealed:</code> refs). Encrypted at rest under a key that only opens after your finger or PIN	Credential theft from disk while you are away or locked
The <code>mcp_only</code> door. With unlock on, LOTR serves only the MCP connection Claude Code started — not a command the assistant runs in its shell	The assistant using LOTR through Bash, around your approvals
Only the authority asks. Every prompt is raised by gt's unlock authority, naming who is asking and for what. A client can request an unlock; it can never claim one	An agent typing "approved" on your behalf
Grants in memory only , bound to one session and revoked on idle (15 min), TTL (8 h), screen lock, sleep, clock rollback, the session or MCP shim ending, and <code>gt_unlock.py lock</code>	A forgotten unlocked session staying open
Admin policy floor. An administrator-owned file that can only tighten	A user or agent loosening an organisation's policy
Fail closed. With unlock on, an unreadable policy, a missing factor, an authority that is down, or a policy edited behind gt's back locks every gated action — never the reverse	"Broken" quietly meaning "open"
No gt crypto store. gt brokers proven stores (1Password, sops+age, Vault, the OS keychains) and uses CryptoKit / DPAPI for sealing; it ships no cipher of its own	Home-made cryptography
Every grant audited. Each unlock, check, secret read and revocation is one line in <code>~/.claude/golden-thread/unlock/audit.jsonl</code> , with its grant id — never a code or a secret value	"What happened while I was unlocked?"

2. How it works



1. A session starts. `gt` registers it with the authority; the LOTR MCP connection Claude Code starts registers itself as that session's one shim.
2. The assistant calls a LOTR tool. LOTR asks the authority whether **this process** (named by the kernel) may use `lotr:<connection>:<tier>`. Locked: the authority asks you — a Touch ID or Windows Hello sheet, then a dialog for your authenticator code — naming the requester and the scope.
3. You approve. The authority checks the signature and the code, grants the session, and LOTR proceeds. The grant id is written into LOTR's audit line for that call.
4. Fifteen idle minutes, eight hours, your screen locking, the lid closing, or the session ending: the grant is gone and the next call asks again.

Reads of LOTR connections can stay open (`read_without_unlock` , on by default); writes and anything gated need the grant. Consent-tier operations (send, merge, delete) still get LOTR's own confirmation, and you can require a Touch ID / Hello approval per operation instead (`gt_unlock.py policy consent platform --window 300` approves consent operations for five minutes, then asks again).

3. What it needs, and how to set it up

Everything below runs from your own terminal. `gt_unlock.py` lives at `~/claude/golden-thread/hooks/gt_unlock.py` (written `gt_unlock.py` below).

MACOS – TOTP + TOUCH ID (L2)

Needs: a Mac with Touch ID, the Xcode Command Line Tools (`xcode-select --install` , so `install.sh` can build the Touch ID helper), and an authenticator app on your phone.

1. Re-run `install.sh`. It builds `~/./claude/golden-thread/bin/gt-presence` from the shipped Swift source and ad-hoc signs it (a Developer ID signed build is provided with releases).
2. `gt_unlock.py enroll touchid` — touch the sensor once. This creates two Secure Enclave keys that only work with your current fingerprints (adding or removing a fingerprint retires them, by design).
3. `gt_unlock.py enroll totp` — scan the QR code, type the code your app shows. The screen is cleared afterwards; the seed is shown only now.
4. `gt_unlock.py recovery` — print ten one-time recovery codes and store them offline.
5. `gt_unlock.py policy enable` — proves your factors work (touch + code), then turns unlock on.

6. `gt_unlock.py status` should say `level L2` . `gt_unlock.py verify` runs the self-check.

WINDOWS — TOTP + WINDOWS HELLO (L2)

Needs: Windows 10/11 with Windows Hello set up (Settings > Accounts > Sign-in options > PIN (Windows Hello); a fingerprint or face reader is optional), and an authenticator app.

1. `gt_unlock.py enroll hello` — Windows Security asks for your PIN or biometric a few times (key creation, a check that the key signs deterministically, which sealing relies on, and a final proof). The Hello dialog shows its own wording ("Making sure it's you"); gt shows the requester and scope in its own prompt.
2. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` as above.

The Hello helper is a Windows PowerShell 5.1 script run with `-ExecutionPolicy Bypass` for its own process only; nothing about your machine's execution policy changes. It is not code-signed: the security rests on the TPM signature gt verifies, not on who signed the script. Organisations that block unsigned scripts sign it with their own certificate. Sealing on Windows needs a desktop logon (user-scope DPAPI is unavailable in an SSH key logon).

LINUX — TOTP (L1), OPTIONALLY ENTRA

Linux has no platform authenticator gt can use, so unlock there is **L1**: it stops agents and accidents, not malware running as you. `gt_unlock.py enroll totp` , `gt_unlock.py recovery` , `gt_unlock.py policy enable` . Without a desktop, codes are typed in the terminal that asked.

MICROSOFT ENTRA ID SIGN-IN (ANY PLATFORM, OPTIONAL)

A fresh browser sign-in (`prompt=login`) can be one of your factors: authorization code + PKCE on a `http://localhost` redirect, the ID token's signature and claims checked by gt (issuer, tenant, audience, nonce bound to the request, sign-in time). You register an app once:

1. Entra admin center > App registrations > New registration. Account types: your organisation only (strongest), any organisation (`organizations`), or work and personal accounts (`common`).
2. Redirect URI: platform *Public client/native (mobile & desktop)*, value `http://localhost` .
3. No client secret. API permissions: Microsoft Graph delegated `openid` , `profile` , `email` only. Leave *Allow public client flows* off.
4. Record the Application (client) ID and Directory (tenant) ID, then: `gt_unlock.py policy sso --client-id <app id> --tenant <tenant id|organizations|common>` and `gt_unlock.py enroll sso` .
5. Enforce MFA in Entra with Conditional Access (authentication strength) on the app; v2 ID tokens do not always say how you signed in. Sign in with Apple is not supported (it cannot redirect to a local address and carries no tenant or MFA claims).

4. What each level protects — and what it does not

`gt_unlock.py status` and `/gt:gt-doctor` (rows `unlock` and `security`) name the level this machine runs at. Nothing here says "secure"; each check passes, fails, or says it could not be checked.

LEVEL	WHAT IT IS	STOPS	DOES NOT STOP
off (default)	Nothing gated	—	Anything
L1 Gate	TOTP (and SSO), no platform factor	The assistant via Bash or other tools, other local agents and scripts, use after the session ends or the screen locks; full audit	Malware running as you: it can read the TOTP seed, the credentials and gt's code
L2 Sealed	Touch ID / Windows Hello required in every unlock; credentials sealed	Credential theft at rest; screen-control agents clicking approve (they cannot touch the sensor)	Malware inside an unlocked window; code patched to capture your next unlock
L3 Separated	Authority and LOTR as a service account, code and policy admin-owned	Use before unlock, policy downgrade, seed and credential reads	An attacker inside your session after you unlock. gt 0.20.0 does not ship L3.

THREAT	L1	L2
The assistant calling LOTR or the authority from Bash	Stopped (<code>mcp_only</code> + grant)	Stopped
Another local agent or script, while locked	Gated, but can read credentials directly	Stopped
Malware running as you, before you unlock	Not stopped	Partly: it cannot decrypt sealed credentials, but can change gt's code for your next unlock
Malware running as you, after you unlock	Not stopped	Not stopped (short idle and TTL limit the window)
A screen-control / accessibility agent	Can click dialogs	Cannot touch the sensor
A keylogger reading your TOTP code	First use wins (each code works once)	Same; use TOTP with a platform factor
A fake prompt racing gt's	Possible	Possible: gt's prompts name the requester and scope
Stolen laptop, locked	Disk encryption + revoke on lock	The Secure Enclave / TPM needs your finger or PIN
Stolen laptop, unlocked and unattended	Until idle / TTL (15 min idle by default)	Same
Identity provider compromise	SSO forged; two factors including a local one mitigate	Same
Root / administrator compromise	Out of scope	Out of scope

Honest limits, in one place:

- At L1 and L2 the authority runs as you, from files you can change. The admin floor and the policy integrity check stop agents and accidents, not someone with your account.
- The settings guard sees Claude Code's Write/Edit tools. A shell command writing `settings.json` is not stopped by the hook; at L2 the credentials it would want are still sealed.
- A brokered secret is only as locked as the weakest of the store, gt's grant, and the way it reaches the program. Environment variables are readable by any process of yours (`ps eww`).
- Unattended jobs name themselves (`GT_JOB`); a process of yours outside a session can claim a job's name. That is why a job may hold only narrow, read-only scopes (below).

5. Locking down more with gt unlock

5.1 THE VAULT: LOCK CHOSEN FOLDERS, NOT THE VAULT

`gt_lock.py add Secrets/aws.md --vault <vault>` encrypts a file with `age` to `aws.md.age` and removes the plaintext; a `.gt-locked` note in the folder says what is locked and how to open it. `gt_lock.py open Secrets/aws.md.age --vault <vault> | less` decrypts to a pipe only (a terminal needs `--show`; a file is always refused) after a `gt:lock:<folder> grant`; `gt_lock.py restore` puts the plaintext back (mode 600); `gt_lock.py status` lists what is locked and how strong each lock is. The identity comes from `--identity`, `$GT_AGE_IDENTITY`, `SOPS_AGE_KEY_FILE`, or `~/.config/sops/age/keys.txt`. Lock before the first commit or sync: plaintext already in git history, sync version history or backups stays there.

- **Strength:** L2 only when every recipient is hardware-held (`age1se1...` Secure Enclave via `age-plugin-se`, `age1yubiky1...`); L1 when any recipient or the identity is a plaintext key such as `~/.config/sops/age/keys.txt` — the usual setup. Never L3: once opened, any process of yours can read the plaintext.
- `core-rules/` and `global-memory/` can never be locked (enforced in code): gt reads them on every turn, and locking them would silently switch Core-rule injection off.
- Why not the whole vault: Obsidian keeps it open, gt's hooks read it on every prompt, and an encrypted volume synced by Dropbox and mounted on two machines can corrupt.
- What others see: Obsidian shows links into a locked file as unresolved; Dropbox, iCloud and git see only `.age` ciphertext (commit only ciphertext); gt's lint, tasks and recall treat a locked folder as absent and say so once.
- An encrypted disk image (macOS sparse bundle) or BitLocker VHDX is a bring-your-own option; once mounted, any process of yours can read it.

5.2 OTHER APPS' CREDENTIALS, THROUGH THE BROKER

Credentials are released only under a grant, and handed over by pipe or a short-lived 600 file, never by environment variable unless you ask for it.

- **git / GitHub publishing:** store the token sealed (`gt_unlock.py seal put github`, value on stdin), map the host in `~/.claude/golden-thread/unlock/credentials.json` (`{"git": [{"protocol": "https", "host": "github.com", "username": "x-access-token", "ref": "sealed:github"}]}`), and set `git config --global credential.https://github.com.helper '!python3 ~/.claude/golden-thread/hooks/gt_unlock.py git-credential'`. Every push then needs a `gt:publish` grant. It is real only if the token exists nowhere else (no plaintext `~/.config/gh/hosts.yml`, no unencrypted SSH key with the same reach).
- **AWS:** `credential_process = python3 ~/.claude/golden-thread/hooks/gt_unlock.py aws-credential --ref sealed:aws` (the sealed value is the JSON AWS expects).
- **Any command:** `gt_unlock.py run --scope gt:publish --secret-file GH_TOKEN_FILE=sealed:github -- <cmd>` hands the command a path to a 600 file deleted when it exits. `--env VAR=REF` puts the value in the environment instead, which is weaker, and gt says so each time.
- `kubectl exec` plugins and docker credential helpers can call `gt_unlock.py secret get REF` the same way (it refuses to print to a terminal).
- Moving an existing credential in: `gt_unlock.py seal migrate file:/abs/path --name github --remove-source` (also `store:` and `keychain:`).

5.3 SSH KEYS

Keep SSH keys in hardware and use them per signature: Secure Enclave keys through an agent such as Secretive (macOS) or a FIDO2 security key (`ssh-keygen -t ed25519-sk`). `gt` does not hold SSH keys; an unencrypted key file in `~/.ssh` is readable by any process of yours whatever `gt` does.

5.4 BRING YOUR OWN SECRET STORE

Refs `sops:` , `op:` , `bw:` , `vault:` and `wincred:` are resolved by the authority under a grant, and `gt_unlock.py status /doctor` label each with its real level:

STORE	LEVEL AGAINST PROCESSES RUNNING AS YOU
Keychain via <code>/usr/bin/security</code> (LOTR's <code>keychain: refs</code>)	L1
Windows Credential Manager / DPAPI (<code>wincred:</code>)	L1
<code>sops</code> + age, identity in a plaintext file	L1
<code>sops</code> + age, identity in the Secure Enclave or a YubiKey	L2
1Password CLI (<code>op:</code>)	L2 (its own biometric unlock and idle limit)
Bitwarden CLI (<code>bw:</code>)	L1 after unlock (the session key sits in the environment or a file)
HashiCorp Vault (<code>vault:</code>)	L1 (0.20.0 does not hold Vault tokens in the authority)
<code>gt</code> 's sealed store (<code>sealed:</code>)	L2

5.5 SCHEDULED JOBS

Nothing is allowed unattended by default. Each job may be given one narrow scope: `gt_unlock.py policy unattended add <job> lotr:<connection>:read` (or `secret:<ref>` for one credential), and the job's definition sets `GT_JOB=<job>` . Write, consent, publish, wildcard and `gt`'s own scopes can never be allowed unattended; jobs are never prompted.

5.6 WHAT GT UNLOCK CANNOT LOCK

- Applications that never ask `gt`: a browser, an editor, any program reading its own config.
- Anything after you unlock, inside the unlocked window, from software already running as you.
- Credentials that also exist somewhere else in plaintext.
- Files outside the folders you lock.

WORKED EXAMPLE (MACOS)

```
gt_unlock.py enroll touchid && gt_unlock.py enroll totp && gt_unlock.py recovery
gt_unlock.py policy enable
printf %s "$GITHUB_TOKEN" | gt_unlock.py seal put github      # then delete the plaintext co
lotr --home ~/.config/gt-lotr/personal add-http github@personal --profile github \
    --base-url https://api.github.com --identity me --auth bearer --token-ref sealed:github
gt_lock.py add Secrets/aws.md --vault ~/Vault
gt_unlock.py verify
```

WORKED EXAMPLE (WINDOWS, POWERSHELL)

```
$u = "$env:USERPROFILE\.claude\golden-thread\hooks\gt_unlock.py"
python $u enroll hello; python $u enroll totp; python $u recovery
python $u policy enable
python $u seal put github      # type the token at the hidden prompt
python $u verify
```

6. Enterprise

- **Admin policy floor:** `/Library/Application Support/gt/unlock-policy.json` (macOS, root-owned with root-owned parents, or delivered by an MDM profile that writes it), `%ProgramData%\gt\unlock-policy.json` (Windows, owned by Administrators/SYSTEM with no user write access) or `HKLM\SOFTWARE\Policies\gt\UnlockPolicy` (a JSON string, for GPO/Intune), `/etc/gt/unlock-policy.json` (Linux). Its existence turns unlock on. It only tightens: more factors, fewer allowed factors, shorter TTL and idle, stricter scopes, `locked_keys` the user cannot change, a pinned Entra tenant. A floor file `gt` cannot verify locks everything.
- The floor is **binding only at L3**, when the enforcing code is also administrator-owned; 0.20.0 runs the authority as the user. Say so in your own policy documents.
- **Signing:** release builds of the macOS helper are Developer ID signed and notarized. The Windows Hello script is unsigned by design; sign it with your own certificate where policy requires signed scripts.
- **Recovery for managed fleets:** keep `admin_only` recovery in your process: reset an enrolment by removing `~/.claude/golden-thread/unlock/` (which also discards sealed credentials) under your own controls.

7. Recovery, and turning it off

- **Lost phone or changed fingerprints:** `gt_unlock.py unlock --recovery` accepts one recovery code as one factor; you then re-enrol (`gt_unlock.py enroll ...`). Each code works once.
- **Lost every factor:** remove `~/.claude/golden-thread/unlock/` and enrol again. Sealed credentials are lost with it — that is what sealed means; keep their originals in your store.

- **Turn it off:** `gt_unlock.py policy disable` (needs your factors) or `gt_settings.py set unlock off`.
Off, every hook behaves exactly as before 0.20.0.
- **Check it:** `gt_unlock.py status`, `gt_unlock.py verify`, `gt_unlock.py audit -n 50`.